

Experiment 8

Student Name: Prateek Verma
Branch: MCA (CC & DEVOPS)
Semester: IInd
Subject Name: Technical Training

UID: 25MCC20062
Section/Group: 25MCC-101/A
Date of Performance: 31/03/2026
Subject Code: 25CAP-652

Title: Stored Procedures in SQL.

1. Aim:

- a. To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

2. Software Requirements:

- a. PostgreSQL (Database Server)
- b. pgAdmin
- c. OS: Windows

3. Objective:

- a. Apply stored procedure concepts for database operations

4. Procedure of the Practical:

- a. Start the system.
- b. Open pgAdmin.
- c. Create and select the database in which you want to perform the experiment.
- d. Establish connection to the database using Alt+Shift+Q.
- e. Run the following queries given in Experiment Steps (5).

Base Table: Employees

	emp_id [PK] integer	emp_name character varying (100)	salary numeric (10,2)	department character varying (50)
1	1	Alice	50000.00	HR
2	2	Bob	60000.00	IT
3	3	Charlie	55000.00	Finance

5. Experiment Steps:

- a. SQL procedure to insert employee data.

```
CREATE OR REPLACE PROCEDURE INSERT_EMPLOYEE_PROC (
    IN P_EMP_ID INT,
    IN P_EMP_NAME VARCHAR(100),
    IN P_SALARY NUMERIC(10,2),
    IN P_DEPARTMENT VARCHAR(50),
    OUT STATUS VARCHAR(50)
) AS $$
BEGIN
    INSERT INTO EMPLOYEES (EMP_ID, EMP_NAME, SALARY, DEPARTMENT)
    VALUES (P_EMP_ID, P_EMP_NAME, P_SALARY, P_DEPARTMENT);
    STATUS := 'Success';
EXCEPTION
    WHEN unique_violation THEN
        STATUS := 'Error: Employee ID already exists';
    WHEN not_null_violation THEN
        STATUS := 'Error: Missing required fields';
    WHEN OTHERS THEN
        STATUS := 'Error: ' || SQLERRM;
END;
$$ LANGUAGE PLPGSQL;
```

CALL INSERT_EMPLOYEE_PROC(4, 'David', 65000, 'Marketing', NULL);

	status character varying
1	Success

	emp_id [PK] integer	emp_name character varying (100)	salary numeric (10,2)	department character varying (50)
1	1	Alice	50000.00	HR
2	2	Bob	60000.00	IT
3	3	Charlie	55000.00	Finance
4	4	David	65000.00	Marketing

b. SQL procedure to update employee salary

```
CREATE OR REPLACE PROCEDURE UPDATE_SALARY_PROC (
    IN P_EMP_ID INT,
    INOUT P_SALARY NUMERIC(20, 3),
    OUT STATUS VARCHAR(20)
) AS $$
DECLARE
    CURR_SAL NUMERIC(20, 3);
BEGIN
    SELECT
        SALARY + P_SALARY INTO CURR_SAL
    FROM
        EMPLOYEES
    WHERE
        EMP_ID = P_EMP_ID;
    IF NOT FOUND THEN RAISE EXCEPTION 'Employee Not Found';
    END IF;
    UPDATE EMPLOYEES
    SET
        SALARY = CURR_SAL
    WHERE
        EMP_ID = P_EMP_ID;
    STATUS := 'Success';
    P_SALARY := CURR_SAL;
    EXCEPTION WHEN OTHERS THEN IF SQLERRM LIKE '%Employee Not Found%'
    THEN STATUS := 'Employee Not Found';
    END IF;
END;
$$ LANGUAGE PLPGSQL;
```

CALL UPDATE_SALARY_PROC(4, 5000.00,null);

	p_salary numeric	status character varying
1	70000.000	Success

	emp_id [PK] integer	emp_name character varying (100)	salary numeric (10,2)	department character varying (50)
1	1	Alice	50000.00	HR
2	2	Bob	60000.00	IT
3	3	Charlie	55000.00	Finance
4	4	David	70000.00	Marketing

- c. SQL procedure to delete employee

```
CREATE OR REPLACE PROCEDURE DELETE_EMPLOYEE_PROC (
    IN P_EMP_ID INT,
    OUT STATUS VARCHAR(50)
) AS $$
DECLARE
    V_DELETED_ROWS INT;
BEGIN
    DELETE FROM EMPLOYEES
    WHERE EMP_ID = P_EMP_ID;
    GET DIAGNOSTICS V_DELETED_ROWS = ROW_COUNT;
    IF V_DELETED_ROWS = 0 THEN
        STATUS := 'Employee Not Found';
    ELSE
        STATUS := 'Success';
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        STATUS := 'Error: ' || SQLERRM;
END;
$$ LANGUAGE PLPGSQL;

CALL DELETE_EMPLOYEE_PROC(4,null);
```

	status character varying			
1	Success			

	emp_id [PK] integer	emp_name character varying (100)	salary numeric (10,2)	department character varying (50)
1	1	Alice	50000.00	HR
2	2	Bob	60000.00	IT
3	3	Charlie	55000.00	Finance

- d. SQL procedure to retrieve a row from the employee table.

```
CREATE OR REPLACE PROCEDURE GET_EMPLOYEE_PROC (
    IN P_EMP_ID INT,
    OUT P_EMP_NAME VARCHAR(100),
    OUT P_SALARY NUMERIC(10,2),
    OUT P_DEPARTMENT VARCHAR(50),
    OUT STATUS VARCHAR(50)
) AS $$
BEGIN
    SELECT EMP_NAME, SALARY, DEPARTMENT
    INTO P_EMP_NAME, P_SALARY, P_DEPARTMENT
    FROM EMPLOYEES
```

```

WHERE EMP_ID = P_EMP_ID;
IF NOT FOUND THEN
    STATUS := 'Employee Not Found';
    P_EMP_NAME := NULL;
    P_SALARY := NULL;
    P_DEPARTMENT := NULL;
ELSE
    STATUS := 'Success';
END IF;
EXCEPTION
    WHEN OTHERS THEN
        STATUS := 'Error: ' || SQLERRM;
        P_EMP_NAME := NULL;
        P_SALARY := NULL;
        P_DEPARTMENT := NULL;
END;
$$ LANGUAGE PLPGSQL;

CALL GET_EMPLOYEE_PROC(4,null,null,null,null);

```

	p_emp_name character varying	p_salary numeric	p_department character varying	status character varying
1	David	65000.00	Marketing	Success

6. Input/Output Details:

- The input for the experiment is the SQL queries mentioned in Experiment Steps (5).
- The output for each query is attached after the query itself.

7. Learning Outcome

- Understanding of how to create procedures in SQL.
- Handling errors in SQL procedures.
- IN, OUT, and INOUT variables in SQL procedures.
- Calling the procedures.